Prompt caching

Copy page

Prompt caching optimizes your API usage by allowing resuming from specific prefixes in your prompts. This significantly reduces processing time and costs for repetitive tasks or prompts with consistent elements.

> This feature is eligible for Zero Data Retention (ZDR). When your organization has a ZDR arrangement, data sent through this feature is not stored after the API response is returned.

There are two ways to enable prompt caching:

- **Automatic caching**: Add a single `cache_control` field at the top level of your request. The system automatically applies the cache breakpoint to the last cacheable block and moves it forward as conversations grow. Best for multi-turn conversations where the growing message history should be cached automatically.
- **Explicit cache breakpoints**: Place `cache_control` directly on individual content blocks for fine-grained control over exactly what gets cached.

The simplest way to start is with automatic caching:

cURL CLI Python TypeScript

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Personalizar Configurações de Cookies

Rejeitar Todos os Cookies Aceitar Todos os Cookies

```
response = client.messages.create(
    model="claude-opus-4-7",
    max_tokens=1024,
    cache_control={"type": "ephemeral"},
    system="You are an AI assistant tasked with analyzing literary works. Your go
    messages=[
        {
            "role": "user",
            "content": "Analyze the major themes in 'Pride and Prejudice'.",
        }
    ],
)
print(response.usage.model_dump_json())
```

With automatic caching, the system caches all content up to and including the last cacheable block. On subsequent requests with the same prefix, cached content is reused automatically.

How prompt caching works

When you send a request with prompt caching enabled:

1. The system checks if a prompt prefix, up to a specified cache breakpoint, is already cached from a recent query.
2. If found, it uses the cached ver
3. Otherwise, it processes the ful

This is especially useful for:

- Prompts with many examples
- Large amounts of context or ba
- Repetitive tasks with consiste
- Long multi-turn conversations

By default, the cache has a 5-minut
each time the cached content is us

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Claude API Docs

For more information, see 1-hour cache duration.

Prompt caching caches the full prefix

Prompt caching references the entire prompt - `tools`, `system`, and `messages` (in that order) up to and including the block designated with `cache_control`.

Pricing

Prompt caching introduces a new pricing structure. The table below shows the price per million tokens for each supported model:

Model	Base Input Tokens	5m Cache Writes	1h Cache Writes	Cache Hits & Refreshes	Output Tokens
Claude Opus 4.7	$5 / MTok	$6.25 / MTok	$10 / MTok	$0.50 / MTok	$25 / MTok
Claude Opus 4.6	$5 / MTok	$6.25 / MTok	$10 / MTok	$0.50 / MTok	$25 / MTok
Claude Opus 4.5	$5 / MTok	$6.25 / MTok	$10 / MTok	$0.50 / MTok	$25 / MTok
Claude Opus 4.1	$15 / MTok	$18.75 / MTok	$30 / MTok	$1.50 / MTok	$75 / MTok
Claude Opus 4	$15 / MTok	$18.75 / MTok	$30 / MTok	$1.50 / MTok	$75 / MTok
Claude Sonnet 4.6	$3 / MTok				
Claude Sonnet 4.5	$3 / MTok				
Claude Sonnet 4	$3 / MTok				
Claude Sonnet 3.7 (deprecated)	$3 / MTok				
Claude Haiku 4.5	$1 / MTok				
Claude Haiku 3.5	$0.80 / MTok				

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Claude API Docs

Claude Opus 3 (deprecated)	$15 / MTok	$18.75 / MTok	$30 / MTok	$1.50 / MTok	$75 / MTok
Claude Haiku 3	$0.25 / MTok	$0.30 / MTok	$0.50 / MTok	$0.03 / MTok	$1.25 / MTok

The table above reflects the following pricing multipliers for prompt caching:

- 5-minute cache write tokens are 1.25 times the base input tokens price
- 1-hour cache write tokens are 2 times the base input tokens price
- Cache read tokens are 0.1 times the base input tokens price

These multipliers stack with other pricing modifiers such as the Batch API discount and data residency. See pricing for full details.

Supported models

Prompt caching (both automatic and explicit) is supported on all active Claude models.

Automatic caching

Automatic caching is the simplest w
`cache_control` on individual cont
level of your request body. The syst
cacheable block.

cURL CLI Python TypeScript

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Claude API Docs

```
response = client.messages.create(
    model="claude-opus-4-7",
    max_tokens=1024,
    cache_control={"type": "ephemeral"},
    system="You are a helpful assistant that remembers our conversation.",
    messages=[
        {"role": "user", "content": "My name is Alex. I work on machine learning.
        {
            "role": "assistant",
            "content": "Nice to meet you, Alex! How can I help with your ML work
        },
        {"role": "user", "content": "What did I say I work on?"},
    ],
)
print(response.usage.model_dump_json())
```

How automatic caching works in multi-turn conversations

With automatic caching, the cache point moves forward automatically as conversations grow. Each new request caches everything up to the last cacheable block, and previous content is read from cache.

Request	Content	Cache behavior
Request 1	System + User(1) + Asst(1) + **User(2)** ◀ cache	Everything written to cache
Request 2	System + User(1) + Asst(1) + User(2) + Asst(2) + **User(3)** ◀ cache	
Request 3	System + User(1) + Asst(1) + User(2) + Asst(2) + User(3) + Asst(3) + **User(4)** ◀ cache	

The cache breakpoint automaticall
you don't need to update any `cach`

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Claude API Docs

By default, automatic caching uses a 5-minute TTL. You can specify a 1-hour TTL at 2x the base input token price:

```
{ "cache_control": { "type": "ephemeral", "ttl": "1h" } }
```

Combining with block-level caching

Automatic caching is compatible with explicit cache breakpoints. When used together, the automatic cache breakpoint uses one of the 4 available breakpoint slots.

This lets you combine both approaches. For example, use explicit breakpoints to cache your system prompt and tools independently, while automatic caching handles the conversation:

```
{
  "model": "claude-opus-4-7",
  "max_tokens": 1024,
  "cache_control": { "type": "ephemeral" },
  "system": [
    {
      "type": "text",
      "text": "You are a helpful assistant.",
      "cache_control": { "type": "ephemeral" }
    }
  ],
  "messages": [{ "role": "user", "content": "What are the key terms?" }]
}
```

What stays the same

Automatic caching uses the same u
thresholds, context ordering requir
same as with explicit breakpoints.

Edge cases

- If the last block already has an
caching is a no-op.

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

- If 4 explicit block-level breakpoints already exist, the API returns a 400 error (no slots left for automatic caching).
- If the last block is not eligible as an automatic cache breakpoint target, the system silently walks backwards to find the nearest eligible block. If none is found, caching is skipped.

> Automatic caching is available on the Claude API and Azure AI Foundry (preview). Support for Amazon Bedrock and Google Vertex AI is coming later.

Explicit cache breakpoints

For more control over caching, you can place `cache_control` directly on individual content blocks. This is useful when you need to cache different sections that change at different frequencies, or need fine-grained control over exactly what gets cached.

Structuring your prompt

Place static content (tool definitions, system instructions, context, examples) at the beginning of your prompt. Mark the end of the reusable content for caching using the `cache_control` parameter.

Cache prefixes are created in the following order: `tools`, `system`, then `messages`. This order forms a hierarchy where each

How automatic prefix checking

You can use just one cache breakpo
automatically find the longest prefi
Understanding how this works help

Three core principles:

1. **Cache writes happen only at y**
   writes exactly one cache entry
   does not write entries for any e

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

2. **Cache reads look backward for entries that prior requests wrote.** On each request the system computes the prefix hash at your breakpoint and checks for a matching cache entry. If none exists, it walks backward one block at a time, checking whether the prefix hash at each earlier position matches something already in the cache. It is looking for prior writes, not for stable content.
3. **The lookback window is 20 blocks.** The system checks at most 20 positions per breakpoint, counting the breakpoint itself as the first. If the system finds no matching entry in that window, checking stops (or resumes from the next explicit breakpoint, if any).

Example: Lookback in a growing conversation

You append new blocks each turn and set `cache_control` on the final block of each request:

- **Turn 1:** 10 blocks, breakpoint on block 10. No prior cache entries exist. The system writes an entry at block 10.
- **Turn 2:** 15 blocks, breakpoint on block 15. Block 15 has no entry, so the system walks back to block 10 and finds the turn-1 entry. Cache hit at block 10; the system processes only blocks 11 through 15 fresh and writes a new entry at block 15.
- **Turn 3:** 35 blocks, breakpoint on block 35. The system checks 20 positions (blocks 35 through 16) and finds nothing. The turn-2 entry at block 15 is one position outside the window, so there is no cache hit. Adding a second breakpoint at block 15 starts a second lookback window there, which finds the turn-2 entry.

Common mistake: Breakpoint on content that changes every request

Your prompt has a large static system
block containing a timestamp and t
block 6:

- **Request 1:** Cache write at bloc
- **Request 2:** The timestamp diff
walks through blocks 5, 4, 3, 2,
positions. No cache hit. You pa
read.

The lookback does not find stable c
entries that prior requests already
`cache_control` to block 5, the last

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

every request, so use an explicit breakpoint on block 5 instead.

Key takeaway: Place `cache_control` on the last block whose prefix is identical across the requests you want to share a cache. In a growing conversation the final block works as long as each turn adds fewer than 20 blocks: earlier content never changes, so the next request's lookback finds the prior write. For a prompt with a varying suffix (timestamps, per-request context, the incoming message), place the breakpoint at the end of the static prefix, not on the varying block.

When to use multiple breakpoints

You can define up to 4 cache breakpoints if you want to:

- Cache different sections that change at different frequencies (for example, tools rarely change, but context updates daily)
- Have more control over exactly what gets cached
- Ensure a cache hit when a growing conversation pushes your breakpoint 20 or more blocks past the last cache write

> **Important limitation:** The lookback can only find entries that earlier requests already wrote. If a growing conversation pushes your breakpoint 20 or more blocks past the last write, the lookback window misses it. Add a second breakpoint closer to that position from the start so a write accumulates there before you need it.

Understanding cache breakpoint costs

Cache breakpoints themselves do

- **Cache writes**: When new cont
  tokens for 5-minute TTL)
- **Cache reads**: When cached co
- **Regular input tokens**: For any u

Adding more `cache_control` brea
same amount based on what conte
give you control over what sections

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Cache limitations

The minimum cacheable prompt length is:

- 4096 tokens for Claude Mythos Preview, Claude Opus 4.7, Claude Opus 4.6, and Claude Opus 4.5
- 2048 tokens for Claude Sonnet 4.6
- 1024 tokens for Claude Sonnet 4.5, Claude Opus 4.1, Claude Opus 4, Claude Sonnet 4, and Claude Sonnet 3.7 (deprecated)
- 4096 tokens for Claude Haiku 4.5
- 2048 tokens for Claude Haiku 3.5 (deprecated)

Shorter prompts cannot be cached, even if marked with `cache_control`. Any requests to cache fewer than this number of tokens will be processed without caching, and no error is returned. To verify whether a prompt was cached, check the response usage fields: if both `cache_creation_input_tokens` and `cache_read_input_tokens` are 0, the prompt was not cached (likely because it did not meet the minimum length requirement).

If your prompt falls just short of the minimum for the model you are using, expanding the cached content to reach the threshold is often worthwhile. Cache reads cost significantly less than uncached input tokens, so reaching the minimum can reduce costs for frequently reused prompts.

For concurrent requests, note that a cache entry only becomes available after the first response begins. If you need cache hits for parallel requests, wait for the first response before sending subsequent requests.

Currently, "ephemeral" is the only s
lifetime.

What can be cached

Most blocks in the request can be c

- Tools: Tool definitions in the `to`
- System messages: Content blo
- Text messages: Content blocks
  assistant turns
- Images & Documents: Content

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Each of these elements can be cached, either automatically or by marking them with `cache_control`.

What cannot be cached

While most request blocks can be cached, there are some exceptions:

- Thinking blocks cannot be cached directly with `cache_control`. However, thinking blocks CAN be cached alongside other content when they appear in previous assistant turns. When cached this way, they DO count as input tokens when read from cache.
- Sub-content blocks (like citations) themselves cannot be cached directly. Instead, cache the top-level block.

  In the case of citations, the top-level document content blocks that serve as the source material for citations can be cached. This allows you to use prompt caching with citations effectively by caching the documents that citations will reference.
- Empty text blocks cannot be cached.

What invalidates the cache

Modifications to cached content can invalidate some or all of the cache.

As described in Structuring your prompt, the cache follows the hierarchy: `tools` → `system` → `messages`. Changes at each level invalidate that level and all subsequent levels.

The following table shows which parts of the cache are invalidated by different types of changes. ✘ indicates that the cache is invalidated, while ✓ indicates that the cache remains valid.

What changes	Tools cache	System cache	M c
Tool definitions	✘	✘	✘
Web search toggle	✓	✘	✘
Citations toggle	✓	✘	✘

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Claude API Docs

Speed setting	✓	✘	✘	Switching between `speed: "fast"` and standard speed invalidates system and message caches
Tool choice	✓	✓	✘	Changes to `tool_choice` parameter only affect message blocks
Images	✓	✓	✘	Adding/removing images anywhere in the prompt affects message blocks
Thinking parameters	✓	✓	✘	Changes to extended thinking settings (enable/disable, budget) affect message blocks
Non-tool results passed to extended thinking requests	✓	✓	Model-specific	On Opus 4.5+ and Sonnet 4.6+, thinking blocks are preserved by default, so the cache remains valid (✓). On earlier Opus/Sonnet models and all Haiku models, all previously-cached thinking blocks are stripped from context, and any messages that follow those thinking blocks are removed from the cache (✘). For more details, see Caching with thinking blocks.

Tracking cache performance

Monitor cache performance using these API response fields, within `usage` in the response (or `message_start` event if streaming):

- `cache_creation_input_tokens`: Number of tokens written to the cache when creating a new entry.
- `cache_read_input_tokens`: Number of tokens retrieved from the cache for this request.
- `input_tokens`: Number of inp
cache (that is, tokens after the

Understanding the token break

The `input_tokens` field repre
breakpoint in your request - not

To calculate total input tokens:

```
total_input_tokens = ca
```

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Claude API Docs

- `cache_creation_input_tokens` = tokens before breakpoint being cached now (writes)
- `input_tokens` = tokens after your last breakpoint (not eligible for cache)

Example: If you have a request with 100,000 tokens of cached content (read from cache), 0 tokens of new content being cached, and 50 tokens in your user message (after the cache breakpoint):

- `cache_read_input_tokens` : 100,000
- `cache_creation_input_tokens` : 0
- `input_tokens` : 50
- **Total input tokens processed**: 100,050 tokens

This is important for understanding both costs and rate limits, as `input_tokens` will typically be much smaller than your total input when using caching effectively.

Caching with thinking blocks

When using extended thinking with prompt caching, thinking blocks have special behavior:

Automatic caching alongside other content: While thinking blocks cannot be explicitly marked with `cache_control`, they get cached as part of the request content when you make subsequent API calls with tool results. This commonly happens during tool use when you pass thinking blocks back to continue the conversation.

Input token counting: When thinking blocks are read from cache, they count as input tokens in your usage metrics. This is important for cost calculation and token budgeting.

Cache invalidation patterns:

- Cache remains valid when only
- On Opus 4.5+ and Sonnet 4.6+, tool-result user content is adde
- On earlier Opus/Sonnet models tool-result user content is adde from context
- This caching behavior occurs e

For more details on cache invalidati

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Claude API Docs

```
Request 1: User: "What's the weather in Paris?"
Response: [thinking_block_1] + [tool_use block 1]

Request 2:
User: ["What's the weather in Paris?"],
Assistant: [thinking_block_1] + [tool_use block 1],
User: [tool_result_1, cache=True]
Response: [thinking_block_2] + [text block 2]
# Request 2 caches its request content (not the response)
# The cache includes: user message, thinking_block_1, tool_use block 1, and tool_

Request 3:
User: ["What's the weather in Paris?"],
Assistant: [thinking_block_1] + [tool_use block 1],
User: [tool_result_1, cache=True],
Assistant: [thinking_block_2] + [text block 2],
User: [Text response, cache=True]
# On earlier Opus/Sonnet and all Haiku models, non-tool-result user block causes
```

On earlier Opus/Sonnet models and all Haiku models, all previous thinking blocks are removed from context at this point. On Opus 4.5+ and Sonnet 4.6+, prior thinking blocks are kept by default and remain part of the cached prefix.

For more detailed information, see the extended thinking documentation.

Cache storage and sharing

⚠ Starting February 5, 2026, promp
organization-level isolation. Cach
between workspaces within the
Azure AI Foundry (preview); Ama
level cache isolation. If you use m
for this change.

- **Organization Isolation**: Caches
organizations never share cach
- **Exact Matching**: Cache hits re
and images up to and including

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

not used.

Best practices for effective caching

To optimize prompt caching performance:

- Start with automatic caching for multi-turn conversations. It handles breakpoint management automatically.
- Use explicit block-level breakpoints when you need to cache different sections with different change frequencies.
- Cache stable, reusable content like system instructions, background information, large contexts, or frequent tool definitions.
- Place cached content at the prompt's beginning for best performance.
- Use cache breakpoints strategically to separate different cacheable prefix sections.
- Place the breakpoint on the last block that stays identical across requests. For a prompt with a static prefix and a varying suffix (timestamps, per-request context, the incoming message), that is the end of the prefix, not the varying block.
- Regularly analyze cache hit rates and adjust your strategy as needed.

Optimizing for different use cases

Tailor your prompt caching strategy to your scenario:

- Conversational agents: Reduce cost and latency for extended conversations, especially those with long instructions or uploaded documents.
- Coding assistants: Improve aut
sections or a summarized versi
- Large document processing: In
in your prompt without increas
- Detailed instruction sets: Share
examples to fine-tune Claude's
in the prompt, but with prompt
including 20+ diverse example
- Agentic tool use: Enhance perf
iterative code changes, where

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

and letting users ask it questions.

Troubleshooting common issues

If experiencing unexpected behavior:

- Ensure cached sections are identical across calls. For explicit breakpoints, verify that `cache_control` markers are in the same locations
- Check that calls are made within the cache lifetime (5 minutes by default)
- Verify that `tool_choice` and image usage remain consistent between calls
- Validate that you are caching at least the minimum number of tokens for the model you are using (see Cache limitations). Length-based caching failures are silent: the request succeeds but both `cache_creation_input_tokens` and `cache_read_input_tokens` will be 0
- Confirm your breakpoint is on a block that stays identical across requests. Cache writes happen only at the breakpoint, and if that block changes (timestamps, per-request context, the incoming message), the prefix hash never matches. The lookback does not find stable content behind the breakpoint; it only finds entries that earlier requests wrote at their own breakpoints
- Verify that the keys in your `tool_use` content blocks have stable ordering as some languages (for example, Swift, Go) randomize key order during JSON conversion, breaking caches

> Changes to `tool_choice` or the presence/absence of images anywhere in the prompt will invalidate the cache, requiring a new cache entry to be created. For more details on cache invalidation, see What invalidates

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

1-hour cache duration

If you find that 5 minutes is too sho
additional cost.

To use the extended cache, include

Claude API Docs

```
  "ttl": "1h"
}
```

The response will include detailed cache information like the following:

Output

```
{
  "usage": {
    "input_tokens": 2048,
    "cache_read_input_tokens": 1800,
    "cache_creation_input_tokens": 248,
    "output_tokens": 503,

    "cache_creation": {
      "ephemeral_5m_input_tokens": 456,
      "ephemeral_1h_input_tokens": 100
    }
  }
}
```

Note that the current `cache_creation_input_tokens` field equals the sum of the values in the `cache_creation` object.

When to use the 1-hour cache

If you have prompts that are used at
used more frequently than every 5
will continue to be refreshed at no

The 1-hour cache is best used in the

- When you have prompts that a
frequently than every hour. Fo
than 5 minutes, or when storin
expect that user may not respo
- When latency is important and
- When you want to improve you
against your rate limit.

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Claude API Docs

Mixing different TTLs

You can use both 1-hour and 5-minute cache controls in the same request, but with an important constraint: Cache entries with longer TTL must appear before shorter TTLs (that is, a 1-hour cache entry must appear before any 5-minute cache entries).

When mixing TTLs, the API determines three billing locations in your prompt:

1. Position `A`: The token count at the highest cache hit (or 0 if no hits).
2. Position `B`: The token count at the highest 1-hour `cache_control` block after `A` (or equals `A` if none exist).
3. Position `C`: The token count at the last `cache_control` block.

> If `B` and/or `C` are larger than `A`, they will necessarily be cache misses, because `A` is the highest cache hit.

You'll be charged for:

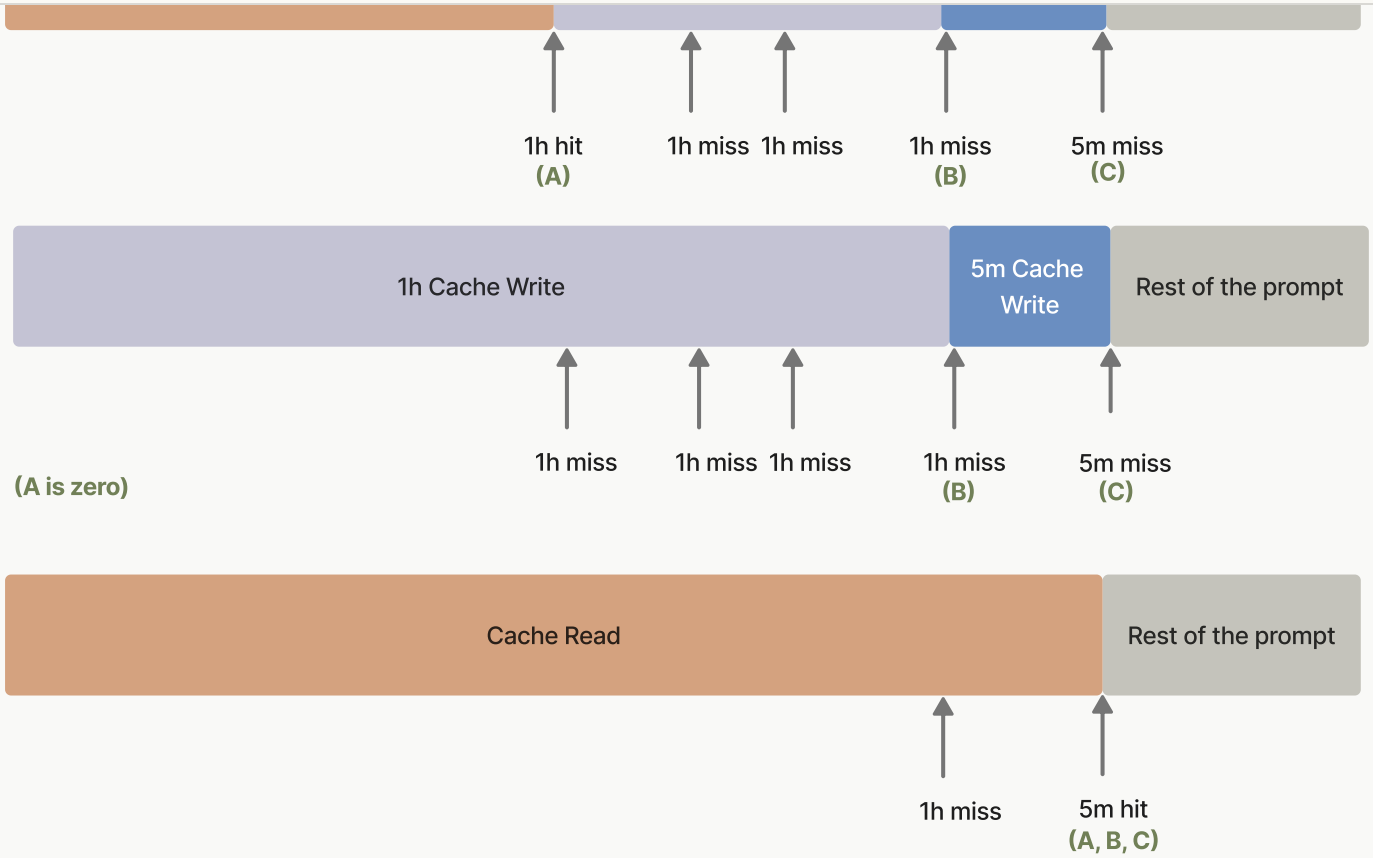
1. Cache read tokens for `A`.
2. 1-hour cache write tokens for `(B - A)`.
3. 5-minute cache write tokens for `(C - B)`.

Here are 3 examples. This depicts the input tokens of 3 requests, each of which has different cache hits and cache misses. Each has a different calculated pricing, shown in the colored boxes, as a result.

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Claude API Docs



Pre-warming the cache

Cache pre-warming lets you load your system prompt or tool definitions into the prompt cache before a user triggers a real request. This eliminates the cache-miss latency penalty on the first user interaction, reducing
applications.

How it works

Set `max_tokens: 0` in your reques
into the model and writing the cach
immediately without generating an
`stop_reason: "max_tokens"`, and

Place the `cache_control` breakpo
request (typically your system pro
message. Otherwise the cache entr

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

placeholder. The placeholder user message can be any string with non-whitespace content (the examples here use `"warmup"`); its content is read during prefill but never answered.

> A pre-warm request incurs a **cache write** charge if the prefix is not already cached, the same as any other request. Check `usage.cache_creation_input_tokens` in the response to confirm a write occurred. Zero output tokens are billed.

cURL CLI Python TypeScript C# Go Java PHP Ruby

```python
client = anthropic.Anthropic()

# Fire this before users arrive to warm the shared system-prompt cache.
prewarm = client.messages.create(
    model="claude-opus-4-7",
    max_tokens=0,
    system=[
        {
            "type": "text",
            "text": "You are an expert software engineer with deep knowledge of c
            "cache_control": {"type": "ephemeral"},
        }
    ],
    messages=[{"role": "user", "content": "warmup"}],
)
print(prewarm.stop_reason)  # "max_tokens"
print(prewarm.content)  # []
print(prewarm.usage)
```

The API returns an empty `content`

Output

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Claude API Docs

```
  "type": "message",
  "role": "assistant",
  "content": [],
  "model": "claude-opus-4-7-20251101",
  "stop_reason": "max_tokens",
  "stop_sequence": null,
  "usage": {
    "input_tokens": 8,
    "cache_creation_input_tokens": 5120,
    "cache_read_input_tokens": 0,
    "cache_creation": {
      "ephemeral_5m_input_tokens": 5120,
      "ephemeral_1h_input_tokens": 0
    },
    "iterations": [
      {
        "input_tokens": 8,
        "output_tokens": 0,
        "cache_read_input_tokens": 0,
        "cache_creation_input_tokens": 5120,
        "cache_creation": {
          "ephemeral_5m_input_tokens": 5120,
          "ephemeral_1h_input_tokens": 0
        },
        "type": "message"
      }
    ],
    "output_tokens": 0,
    "service_tier": "standard",
    "inference_geo": "global"
  }
}
```

Typical usage pattern

Fire a pre-warm request when your
real user requests after the pre-wa

Python

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Claude API Docs

```
SYSTEM_PROMPT = [
    {
        "type": "text",
        "text": "You are an expert software engineer with deep knowledge of distr
        "cache_control": {"type": "ephemeral"},
    }
]


def prewarm_cache() -> None:
    """Call this at application startup or on a scheduled interval."""
    client.messages.create(
        model="claude-opus-4-7",
        max_tokens=0,
        system=SYSTEM_PROMPT,
        messages=[{"role": "user", "content": "warmup"}],
    )


def respond(user_message: str) -> anthropic.types.Message:
    """The real user request; benefits from a warm cache."""
    return client.messages.create(
        model="claude-opus-4-7",
        max_tokens=1024,
        system=SYSTEM_PROMPT,
        messages=[{"role": "user", "content": user_message}],
    )


# Warm the cache before any us
prewarm_cache()

# Later, when the user submit
response = respond("How do I
print(response.content[0].tex
```

Keep in mind that the cache TTL sti
warm request at least every 5 minu
user requests, use the 1-hour cache

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Limitations

- `stream: true`
- Extended thinking (`thinking.type: "enabled"`)
- Structured outputs (`output_config.format`)
- `tool_choice` of `{"type": "tool", ...}` or `{"type": "any"}`

`max_tokens: 0` is also rejected inside a Message Batches request. Pre-warming targets time-to-first-token, which does not apply to batch processing, and a cache entry written during batch processing would likely expire before the follow-up request runs.

Replacing the max_tokens=1 workaround

Before `max_tokens: 0` was available, some applications used `max_tokens: 1` warm-up calls to achieve the same effect. The `max_tokens: 0` approach is preferred: no output is produced, so there is no single-token reply to discard, no output tokens are billed, and the intent of the request is unambiguous.

Prompt caching examples

To help you get started with prompt caching, the prompt caching cookbook provides detailed examples and best practices.

The following code snippets showcase various prompt caching patterns. These examples demonstrate how to implement caching in different scenarios, helping you understand the practical applications of this featur

- **Large context caching examp**
- **Caching tool definitions**
- **Continuing a multi-turn conv**
- **Putting it all together: Multip**

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Claude API Docs

Prompt caching (both automatic and explicit) is ZDR eligible. Anthropic does not store the raw text of your prompts or Claude's responses.

KV (key-value) cache representations and cryptographic hashes of cached content are held in memory only and are not stored at rest. Cached entries have a minimum lifetime of 5 minutes (standard) or 60 minutes (extended), after which they are promptly, though not immediately, deleted. Cache entries are isolated between organizations.

For ZDR eligibility across all features, see API and data retention.

FAQ

- **Do I need multiple cache breakpoints or is one at the end sufficient?**
- **Do cache breakpoints add extra cost?**
- **How do I calculate total input tokens from the usage fields?**
- **What is the cache lifetime?**
- **How many cache breakpoints can I use?**
- **Is prompt caching available fo**
- **How does prompt caching wo**
- **How do I enable prompt cach**
- **Can I use prompt caching wit**

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Claude API Docs

- Can I manually clear the cache?
- How can I track the effectiveness of my caching strategy?
- What can break the cache?
- How does prompt caching handle privacy and data separation?
- Can I use prompt caching with the Batches API?
- Why am I seeing the error `AttributeError: 'Beta' object has no attribute 'prompt_caching'` in Python?
- Why am I seeing 'TypeError: Cannot read properties of undefined (reading 'messages')'?

Esta página foi útil?

Claude API Docs

Solutions

AI agents

Code modernization

Coding

Customer support

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.

Claude API Docs

Government

Life sciences

Partners

Amazon Bedrock

Google Cloud's Vertex AI

Events

Powered by Claude

Service partners

Startups program

Company

Anthropic

Careers

Economic Futures

Research

News

Responsible Scaling Policy

Security and compliance

Transparency

Help and security

Availability

Status

Support

Discord

Terms and policies

Privacy policy

Responsible disclosure policy

Terms of service: Commercial

Terms of service: Consumer

Usage policy

Configurações de cookies

Usamos cookies para fornecer e melhorar nossos serviços, analisar o uso do site e, se você concordar, personalizar sua experiência e comercializar nossos serviços para você. Você pode ler nossa Política de Cookies aqui.